

UNIVERSIDAD DEL VALLE DE GUATEMALA

CC3067 – Redes de Computadoras

Sección 10

Ing. Kevin Velásquez



Proyecto No. 1

Uso de un protocolo existente

Angie Nadissa Vela López, 23764

Guatemala, 22 de septiembre de 2026

1. Introducción

En este proyecto se desarrolló un chatbot de consola que funciona como anfitrión de varios servidores MCP. El chatbot se comunica con un modelo de lenguaje mediante la API de Anthropic y, a través de clientes MCP implementados manualmente, puede utilizar servidores para interactuar con el sistema de archivos, Git y una aplicación propia orientada al área de farmacias.

Uno de los principales objetivos del proyecto fue comprender el funcionamiento del protocolo más allá de utilizar una implementación existente. Por esta razón, la comunicación JSON-RPC y el ciclo de vida de MCP fueron implementados manualmente, sin utilizar SDKs que abstraieran el funcionamiento del protocolo.

El servidor desarrollado para este proyecto corresponde a un caso de uso de una cadena de farmacias. Este servidor permite evaluar síntomas, buscar medicamentos, revisar interacciones y alergias, consultar el inventario de diferentes sucursales y crear órdenes de compra, incluyendo reglas para evitar la compra de medicamentos de prescripción cuando no se proporciona una receta.

El Model Context Protocol (MCP) es un protocolo abierto propuesto por Anthropic en noviembre de 2024 que estandariza la forma en que los modelos de lenguaje (LLMs) acceden a herramientas externas. Antes de MCP, cada proveedor de LLM definía su propio mecanismo de integración de herramientas, lo que obligaba a reimplementar cada herramienta para cada modelo. MCP resuelve este problema separando claramente tres roles: el servidor (expone herramientas), el cliente (mantiene la conexión con un servidor) y el anfitrión (la aplicación de IA que coordina varios clientes).

Este documento describe el proyecto completo, en sus dos entregas: el chatbot general (conexión con el LLM, contexto de conversación y bitácora de interacciones MCP), el uso de los servidores MCP oficiales de Filesystem y Git, el servidor MCP propio para el caso de uso de farmacia ejecutado tanto de forma local como remota en Google Cloud Run, el análisis de la comunicación con Wireshark, y la interfaz de usuario web construida como extra.

2. Objetivos

2.1 Objetivo General

Implementar un chatbot capaz de utilizar múltiples herramientas mediante el Model Context Protocol (MCP), desarrollando manualmente la comunicación basada en JSON-RPC y conectando el sistema con un Large Language Model mediante su API.

2.2 Objetivos específicos

- Implementar un protocolo en base a los estándares.
- Comprender el propósito del protocolo MCP.
- Comprender cómo funcionan los servicios del protocolo MCP.
- Implementar servidores MCP locales y remotos
- Comprender como interactuar con un LLM a nivel de la API.

3. Marco teórico

3.1 Large Language Models

Los Large Language Models son modelos de inteligencia artificial entrenados con grandes cantidades de información para procesar y generar lenguaje natural. Una de sus principales

características es su capacidad para interpretar las instrucciones proporcionadas por un usuario y generar respuestas coherentes de acuerdo con el contexto disponible.

Sin embargo, un LLM no tiene acceso directo a todos los sistemas externos que una aplicación podría necesitar. Por ejemplo, el modelo no puede consultar directamente el inventario de una farmacia, modificar un archivo local o ejecutar un comando de Git únicamente por conocer la respuesta correcta. Para realizar este tipo de acciones necesita utilizar herramientas.

En este proyecto, el LLM funciona como el componente encargado de interpretar las solicitudes del usuario y decidir cuándo necesita utilizar una herramienta. Las acciones reales son ejecutadas por los servidores MCP.

3.2 Model Context Protocol

Model Context Protocol (MCP) es un protocolo diseñado para facilitar la comunicación entre aplicaciones que utilizan modelos de lenguaje y herramientas externas.

El protocolo define una arquitectura en la que participan principalmente tres componentes: el anfitrión, el cliente y el servidor.

El anfitrión es la aplicación que utiliza el modelo de lenguaje y coordina las diferentes herramientas disponibles. En este proyecto, el chatbot de consola funciona como anfitrión.

El cliente MCP es el componente encargado de mantener la comunicación entre el anfitrión y un servidor específico. Un mismo anfitrión puede tener varios clientes, uno por cada servidor al que se conecta.

Finalmente, el servidor MCP proporciona las herramientas que pueden ser utilizadas por el modelo. En este proyecto se utilizaron servidores existentes para Filesystem y Git, además de un servidor propio para el caso de uso de farmacia.

Esta separación permite que el servidor de una herramienta no dependa directamente del modelo de lenguaje utilizado por el anfitrión.

3.3 JSON-RPC

MCP utiliza JSON-RPC para estructurar la comunicación entre clientes y servidores.

JSONRPC permite representar diferentes tipos de mensajes utilizando objetos JSON.

En el proyecto se implementaron manualmente los principales tipos de mensajes necesarios para la comunicación:

- Solicitudes.
- Respuestas.
- Notificaciones.
- Respuestas de error.

Cada solicitud posee un identificador que permite relacionar posteriormente la respuesta con la operación que la originó.

Para la comunicación mediante stdio, cada mensaje JSON-RPC se transmite como una línea independiente. De esta forma, el cliente puede enviar una solicitud y posteriormente esperar la respuesta correspondiente.

Una de las decisiones importantes del proyecto fue implementar esta comunicación directamente, en lugar de utilizar una biblioteca que ocultara el funcionamiento de JSONRPC

o de MCP. Esto permitió comprender de manera más directa cómo se estructura y transporta la información.

4. Caso de uso implementado

El caso de uso de industria elegido es un asistente conversacional para una cadena de farmacias. A partir de los síntomas que describe el usuario, el asistente evalúa la gravedad del cuadro, identifica señales de alarma que requieren atención médica inmediata, recomienda medicamentos de venta libre (OTC) apropiados, verifica alergias, consulta existencias por sucursal y permite generar un pedido. Los medicamentos que requieren receta se rechazan a menos que se proporcione un identificador de receta válido.

Esta es una funcionalidad sensible por naturaleza (el asistente literalmente recomienda un tratamiento) por lo que las reglas de seguridad (señales de alarma, restricción de medicamentos con receta, disclaimer médico) se implementaron dentro del propio servidor MCP y no únicamente como instrucciones en el prompt del modelo, de forma que se cumplen sin depender de que el LLM las respete.

5. Arquitectura del sistema

5.1 Arquitectura por capas

El proyecto está organizado en capas con dependencias que solo apuntan hacia abajo, es decir, cada capa únicamente conoce a la que tiene inmediatamente debajo. Esto permite probar y razonar sobre cada capa de forma aislada, y hace evidente en el propio código dónde vive cada responsabilidad del protocolo.

Capa	Responsabilidad	No conoce
src/protocol/	JSON-RPC 2.0 puro: mensajes, errores, framing	MCP ni el LLM
src/mcp/	Ciclo de vida MCP y transportes (stdio)	El LLM
src/llm/	Cliente crudo de la API de mensajes de Anthropic	MCP
src/host/	Registro de tools, conversación, agent loop, orquestación	La terminal
src/observability/	Bitácora y renderizado de frames MCP	—
src/servers/pharmacy/	Servidor MCP propio: dispatcher y lógica de dominio	El transporte usado
src/main.py	REPL de terminal	JSON-RPC directamente

5.2 Stack tecnológico

- Python 3.11 con asyncio para toda la concurrencia (subprocesos, E/S de red).
- httpx para las llamadas HTTP crudas a la API de Anthropic, sin el SDK oficial de Anthropic.
- Sin ninguna dependencia de MCP/FastMCP: JSON-RPC y el ciclo de vida MCP están implementados desde cero en src/protocol/ y src/mcp/.
- pytest y pytest-asyncio para la suite de pruebas automatizadas.
- Los servidores MCP oficiales (Filesystem, Git) se ejecutan como procesos hijos vía npx y pip respectivamente; su implementación interna no es parte de este proyecto.

5.3 Estructura de directorios

```
src/  
  protocol/      JSON-RPC 2.0 escrito a mano (mensajes, errores, framing)  
  mcp/           Ciclo de vida MCP sobre JSON-RPC: cliente, modelos, transportes  
  llm/           Cliente crudo de la API de mensajes de Anthropic  
  observability/ Bitácora de frames MCP (JSONL) y renderizado en terminal  
  host/         El anfitrión: configuración, registro de tools, conversación, agent loop  
  servers/pharmacy/ El servidor MCP propio (lógica de dominio, datos, dispatcher)  
  main.py        Punto de entrada: REPL de terminal  
  tests/         Pruebas unitarias e integración  
  docs/          Especificación del servidor, escenarios de demo, este informe  
  config/        Registro declarativo de servidores MCP (servers.json)
```

6. Funcionalidades implementadas

Esta sección demuestra los requisitos del enunciado correspondientes al funcionamiento general del chatbot y a la primera parte de servidores MCP.

6.1 Conexión con un LLM a nivel de su API

Implementado en `src/llm/anthropic_api.py`. La clase `AnthropicClient` construye a mano el cuerpo JSON de la solicitud (`model`, `max_tokens`, `system`, `messages`, `tools`) y lo envía con `httpx.AsyncClient` directamente a `https://api.anthropic.com/v1/messages`, usando los encabezados `x-api-key` y `anthropic-version`. No se usa el SDK `anthropic`, por lo que la forma exacta de la solicitud y la respuesta permanece visible en el código. Se verificó con la pregunta `¿Quién fue Alan Turing?` usando una clave de API real, obteniendo una respuesta correcta con `stop_reason` `“end_turn”`.

6.2 Mantener contexto en una sesión

Implementado en `src/host/conversation.py`. La clase `Conversation` acumula el historial completo de mensajes en el formato que espera la API de Anthropic (role + bloques de contenido) y lo recorta a un máximo configurable de mensajes antes de cada envío, para no hacer crecer la ventana de contexto sin límite. Se verificó igualmente con la pregunta ¿Quién fue Alan Turing? seguida de ¿En qué año nació?, fue respondida correctamente sin repetir el nombre, demostrando que el historial se mantiene entre turnos.

6.3 Bitácora de todas las interacciones con servidores MCP

Implementado en `src/observability/mcp_log.py` y `src/observability/render.py`. Cada `MCPClient` recibe un callback `on_frame` que se invoca por cada mensaje JSON-RPC enviado o recibido (solicitudes, respuestas y notificaciones, de los tres servidores conectados), antes de cualquier otro procesamiento. Cada frame se agrega a un buffer en memoria (los últimos 500) y se escribe como una línea JSON en `logs/mcp-<timestamp>.jsonl`. El comando `/log` del REPL muestra los últimos `n` frames capturados, con dirección, servidor, tipo de mensaje (solicitud/respuesta/notificación/error) y el cuerpo JSON completo. Se verificó ejecutando el REPL real, encadenando `/servers`, `/tools` y `/log` tras una conversación con tool calling.

6.4 Servidores MCP locales oficiales (Filesystem y Git)

El host se conectó exitosamente a los dos servidores MCP oficiales de Anthropic vía transporte `stdio` (`config/servers.json`), el `Filesystem MCP server` (`modelcontextprotocol/server-filesystem`, lanzado con `npx`) y el `Git MCP server` (`mcp-servergit`, instalado con `pip`). Ambos exponen su catálogo completo de herramientas al iniciar

(14 y 12 respectivamente), confirmando que el ciclo de vida MCP implementado a localmente funciona con implementaciones reales y el servidor propio.

Respecto al escenario, en las pruebas se le pidió al asistente crear un archivo README.md dentro de un repositorio, agregarlo al índice y confirmarlo (realizar un commit). El asistente encadenó correctamente todos los procesos, se verificó directamente con git log en el repositorio de prueba. Una limitación que encontré es que la versión publicada de mcpserver-git no expone una herramienta git_init, por lo que el repositorio debe existir de antemano. Por ende, el chatbot no puede crear el repositorio por sí mismo usando únicamente estos dos servidores oficiales. El escenario de demostración se ajustó en consecuencia y queda documentado en docs/demo-scenarios.md.

6.5 Servidor MCP propio con caso de uso de industria

El servidor Pharmacy MCP (src/servers/pharmacy/) está completamente implementado, probado y documentado para ejecución local sobre stdio. Su especificación completa se detalla a continuación y en docs/pharmacy-mcp-spec.md.

7. Especificación del servidor Pharmacy MCP

El servidor se ejecuta con python -m src.servers.pharmacy.stdio_server. Su dispatcher (src/servers/pharmacy/core/dispatcher.py) es independiente del transporte, recibe un mensaje JSON-RPC decodificado y devuelve la respuesta correspondiente, sin saber si llegó por stdio o por HTTP. Esto permite que el mismo servidor se exponga de forma remota en la segunda entrega sin reescribir la lógica de negocio.

7.1 Herramientas (tools) expuestas

Tool	Descripción
assess_symptoms	Evalúa síntomas reportados; devuelve condición probable, severidad, señales de alarma y clases terapéuticas sugeridas.
search_medications	Busca en el catálogo por texto libre y/o clase terapéutica.
get_medication_details	Ficha técnica completa de un SKU: dosis, contraindicaciones, efectos secundarios, precio.
check_interactions	Verifica interacciones fármaco-fármaco y conflictos de alergia contra medicamentos actuales.
check_stock	Consulta precio y existencias por sucursal o en todas las sucursales.
create_order	Genera un pedido; rechaza medicamentos con receta sin un prescription_id.
get_order_status	Consulta el estado de un pedido por su identificador.

7.2 Reglas de seguridad

- Solo se recomiendan medicamentos de venta libre, nunca se despacha un medicamento con receta sin un prescription_id.
- Cualquier señal de alarma (dolor de pecho, dificultad para respirar, sangre, desmayo, confusión, síntomas en menores de 2 años, más de 10 días de evolución) fuerza severity: “urgent” y una recomendación explícita de buscar atención médica inmediata, en lugar de un medicamento.
- Toda evaluación de síntomas incluye un disclaimer explícito indicando que no es un diagnóstico médico.
- Estas reglas están implementadas en el dominio del servidor (core/domain/triage.py y core/domain/orders.py), no únicamente en el prompt del sistema, por lo que se cumplen independientemente de lo que el modelo decida hacer.

7.3 Modelo de datos

Los datos del catálogo viven en `src/servers/pharmacy/data/` y se cargan una vez por proceso, 10 medicamentos (8 de venta libre, 2 con receta), 6 mapeos de síntomas a condiciones más una lista de palabras clave de alarma, reglas de interacción fármaco-fármaco etiquetadas como “warning” o “blocking”, y 3 sucursales con existencias por SKU. Los pedidos se mantienen en memoria durante la vida del proceso y descuentan existencias de la sucursal correspondiente al confirmarse.

7.4 Ejemplo de intercambio JSON-RPC

Solicitud (`tools/call`):

```
{"jsonrpc": "2.0", "id": 4, "method": "tools/call", "params":  
  {"name": "check_interactions", "arguments":  
    {"candidate_skus": ["IBU400"], "current_medications": ["warfarin"]}}
```

Respuesta:

```
{"jsonrpc": "2.0", "id": 4, "result": {"content": [{"type": "text", "text":  
  "\blocking": [...] }"],  
  "{\n    \"safe_to_recommend\": false,\n    \"isError\": false}}
```

`isError` permanece en `false`: la verificación se ejecutó correctamente, solo que el resultado indica que la combinación no es segura. `isError: true` se reserva para fallos de ejecución de la herramienta (SKU desconocido, falta de receta, existencias insuficientes), siguiendo la convención de MCP de que esos casos son resultados normales que el modelo debe ver y ante los que debe reaccionar, no errores de transporte JSON-RPC.

8. Servidor MCP remoto

El mismo servidor Pharmacy MCP descrito en la sección 7 del informe, se expuso de forma remota sobre Streamable HTTP y se desplegó en Google Cloud Run. El chatbot usa el servidor remoto exactamente igual que el local, es decir, utiliza el mismo Dispatcher, las mismas siete tools y las mismas reglas de seguridad. Únicamente cambia el transporte.

8.1 Transporte Streamable HTTP

El script `src/mcp/transport/streamable_http.py` implementa a mano el lado cliente del transporte Streamable HTTP de MCP (sin SDK). Cada mensaje JSON-RPC se envía como un POST independiente, y la respuesta se interpreta como JSON plano o como un cuerpo `text/event-stream` (SSE) con eventos data.

Del lado del servidor, `src/servers/pharmacy/http_server.py` expone el mismo Dispatcher que usa `stdio_server.py` detrás de una app Starlette mínima:

Método	Ruta	Propósito
POST	/mcp	Enviar un mensaje JSON-RPC (solicitud o notificación)
DELETE	/mcp	Terminar la sesión MCP actual
GET	/status	Verificación de vida (liveness check), responde 200 "ok"

El manejo de sesión sigue la especificación, la respuesta a `initialize` incluye un header `Mcp-Session-Id` (un UUID nuevo) que el cliente debe reenviar en cada solicitud posterior junto con `MCP-Protocol-Version 2025-06-18`. Una solicitud con un `Mcp-Session-Id` desconocido recibe 404, que es la señal para que el cliente vuelva a inicializar.

Además, el endpoint de liveness se cambió a `/status`, porque `/healthz` es un path reservado a nivel de la infraestructura de Google (Google Frontend / GFE) para chequeos de salud internos, por ende, cualquier petición externa a esa ruta exacta nunca llega al contenedor, sin importar que la aplicación tenga la ruta definida correctamente.

8.2 Despliegue en Google Cloud Run

El contenedor se construye con `src/servers/pharmacy/Dockerfile` a partir de `python:3.11-slim`, copiando únicamente el subárbol que `http_server.py` realmente importa (`src/protocol/`, los módulos de constantes/modelos de MCP que usa, y el propio servidor de farmacia) más su propio `requirements.txt`, sin `httpx` ni el transporte `stdio`, que no se usan en el endpoint remoto. El archivo `cloudbuild.yaml` define el build para Cloud Build, de forma que no hace falta tener Docker instalado en la máquina que despliega.

```
gcloud builds submit --config cloudbuild.yaml .

gcloud run deploy pharmacy-mcp \
  --image gcr.io/PROJECT_ID/pharmacy-mcp \
  --region us-central1 \
  --allow-unauthenticated \
  --port 8080
```

El servicio quedó publicado en `https://pharmacy-mcp-65182573605.us-central1.run.app/mcp`, con `--allow-unauthenticated` porque este es un proyecto académico, no un patrón a seguir para un servicio con datos reales. El `config/servers.json` define la entrada `pharmacy-remote` apuntando a esa URL a través de la variable de entorno `PHARMACY_REMOTE_URL` (resuelta en `src/host/config.py` con un helper que expande referencias `${VAR}` completas sin interpretar texto arbitrario), y en esta entrega `pharmacy-remote` queda habilitado por defecto en lugar de la versión local, ya que es lo que se califica en esta fase.

8.3 Dificultades del despliegue remoto

- El servicio de Cloud Run estaba configurado como "Require authentication" en lugar de "Allow public access" en la consola, lo que hacía que toda petición no autenticada recibiera la misma página genérica de error de Google en vez de una respuesta de la aplicación. Se corrigió desde Security > Authentication en la consola de Cloud Run.
- El path /healthz resultó estar reservado por la infraestructura de Google, se renombró a /status en el código y se redesplegó.
- El despliegue requirió instalar el CLI de gcloud localmente y autenticarse con gcloud auth login y gcloud config set project, además de identificar el project id correcto entre varios proyectos de prueba existentes en mi cuenta.

9. Interfaz de usuario

Además del REPL de terminal (src/main.py), el proyecto incluye una interfaz de chatbot web (src/web/) que envuelve el mismo Host y AgentLoop sin tocar la capa MCP/JSON-RPC. Se ejecuta con `python -m src.web.server` (Starlette + un WebSocket en /ws) y sirve un frontend estático (HTML/CSS/JS sin frameworks) en src/web/static/.

Es un único chat cuyo tema visual cambia en vivo según qué servidor MCP está respondiendo en ese momento de la conversación: un tema celeste y blanco, minimalista, para las herramientas de filesystem y git; y un tema inspirado en branding de la farmacia Meykos, que se activa apenas el modelo invoca una tool de pharmacy-remote. El cambio de tema se dispara desde el mismo hook on_frame que ya usaba el /log de la terminal. Cada vez que el host envía un tools/call, el servidor web reenvía ese evento por el WebSocket junto con el nombre del servidor, y el frontend decide el tema según ese nombre (cualquier servidor que

empiece con "pharmacy" activa el tema de farmacia). Las respuestas del asistente se renderizan con un subconjunto seguro de Markdown (encabezados, negritas, código, listas) en vez de mostrar la sintaxis cruda que devuelve el modelo.

Esta interfaz se pensó siguiendo principios básicos de HCI, los dos temas comparten la misma estructura y jerarquía visual (para no confundir al usuario cuando cambia), se evitó el uso de colores externos a la paleta de color escogida, y el estado de cada servidor conectado (nombre y cantidad de tools) queda visible en todo momento en la franja superior, de forma que el usuario sabe en todo momento con qué herramientas cuenta el asistente.

10. Análisis de tráfico con Wireshark

Esta sección detalla como se capturó la comunicación entre el anfitrión y el servidor MCP remoto, y la clasificación de los mensajes JSON-RPC según su tipo. El archivo de captura completo se entrega en captures/pharmacy-remote-capture.pcapng, junto con las capturas de pantalla usadas como evidencia en esta sección.

10.1 Metodología: descifrado de TLS

El servidor remoto usa HTTPS (TLS 1.3), por lo que una captura sin más solo muestra registros "Application Data" cifrados, sin poder ver el contenido JSON-RPC. Para poder inspeccionar el tráfico real se agregó soporte para volcar las claves de sesión TLS al formato estándar SSLKEYLOGFILE en src/mcp/transport/streamable_http.py, cuando esa variable de entorno está definida, el cliente httpx crea su contexto SSL con keylog_filename apuntando a ese archivo. En cualquier otra ejecución no cambia nada del comportamiento por defecto.

```
$env:SSLKEYLOGFILE = "...\\MCP-Host-Chatbot\\sslkeys.log" python -m src.main
```

En Wireshark, Edit > Preferences > Protocols > TLS > (Pre)-Master-Secret log filename se configuró apuntando a ese mismo archivo, lo que permite a Wireshark reconstruir y descifrar la sesión TLS en tiempo real durante la captura. El archivo sslkeys.log nunca se sube al repositorio porque son secretos de sesión válidos para descifrar esa captura específica.

10.2 Clasificación de los mensajes capturados

Con la captura descifrada, Wireshark reclasifica los segmentos TLS reensamblados como paquetes HTTP con su cuerpo JSON legible. Sobre una conversación real generada al pedirle al chatbot una recomendación (tengo goteo nasal y dolor de garganta...), los mensajes se clasifican así:

Categoría	Mensajes observados
Sincronización (transporte)	Three-way handshake TCP: SYN, SYN ACK, ACK (puerto 443 del servidor, IP 34.143.78.2).
Sincronización (aplicación/seguridad)	Handshake TLS 1.3: Client Hello (con SNI=pharmacy-mcp-65182573605.us-central1.run.app), Server Hello + Change Cipher Spec, Encrypted Extensions + Certificate + Certificate Verify + Finished, y el Finished del cliente.
Solicitud (request)	Cada POST /mcp con un cuerpo JSON-RPC con id: initialize, notifications/initialized (sin id, es una notificación), tools/list, y varios tools/call (assess_symptoms, search_medications, check_stock, check_interactions).
Respuesta (response)	HTTP/1.1 200 OK con content-type text/event-stream y un evento data: {"jsonrpc":"2.0","id":...,"result":...} por cada solicitud; HTTP/1.1 202 Accepted con cuerpo vacío para la notificación notifications/initialized, que por definición no espera respuesta.

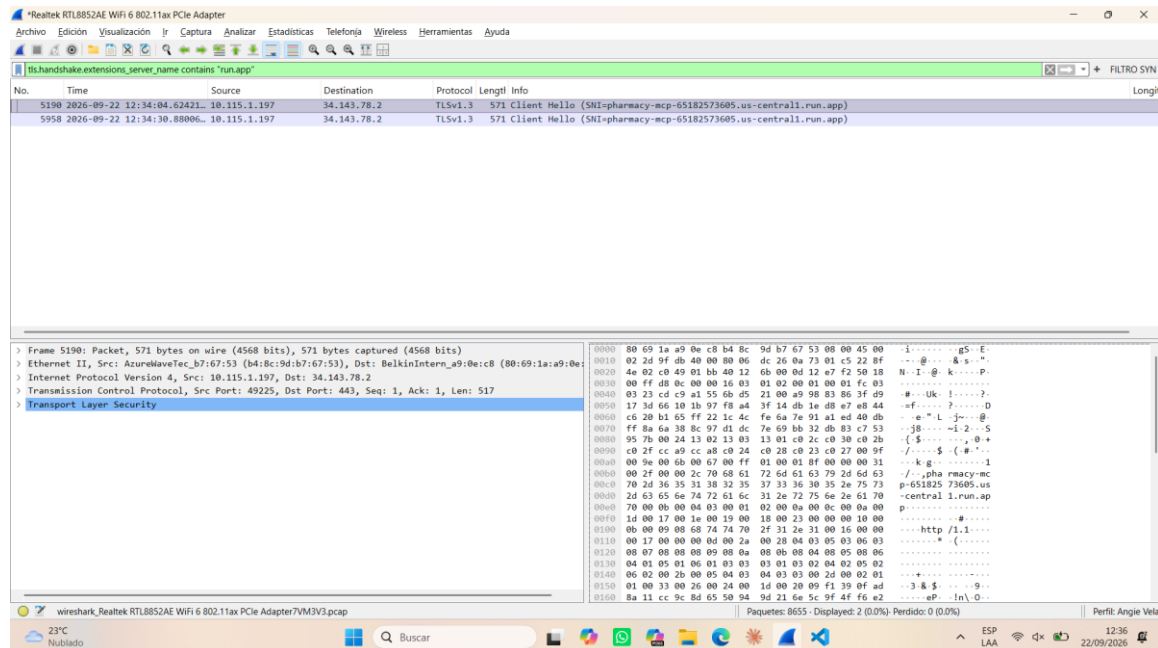


Figura 1. Handshake TCP y TLS capturados (paquetes 5187-5199) contra 34.143.78.2:443, antes del descifrado por paquete individual.

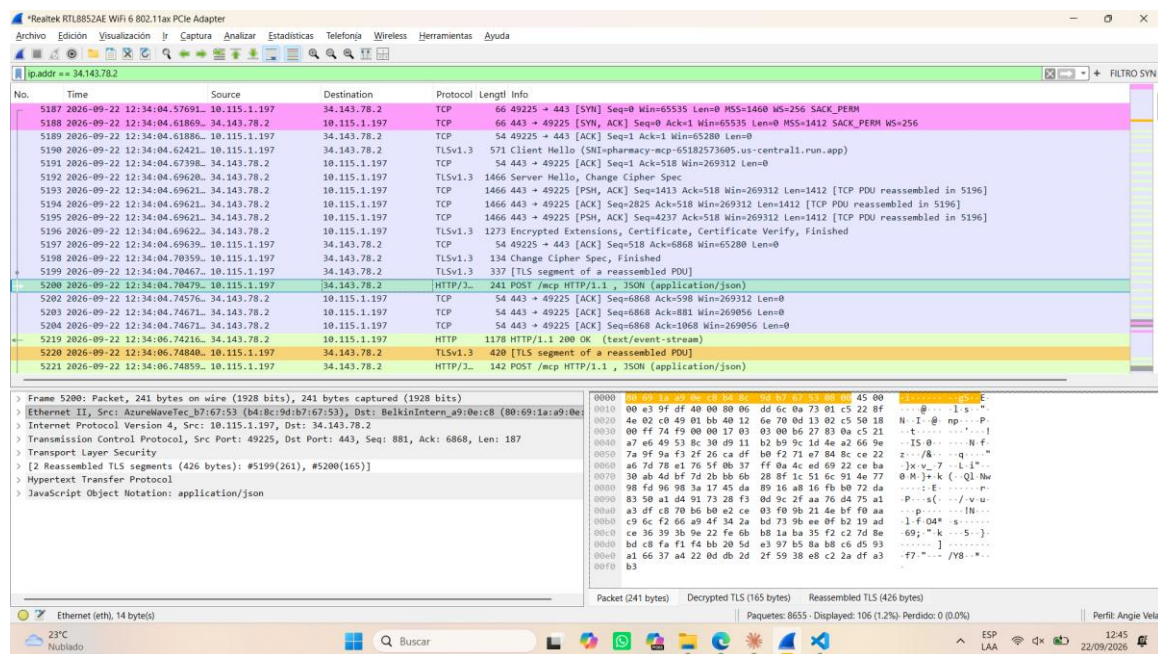


Figura 2. Paquete 5200 ya reensamblado y descifrado: se ven las seis capas completas (Ethernet, IPv4, TCP, TLS, HTTP, JSON) para el POST /mcp del método initialize.

10.3 Análisis por capas del modelo OSI / TCP-IP

Capa	Qué se observó en la captura
Enlace	Cada trama muestra direcciones MAC origen y destino (Ethernet II: AzureWaveTec como origen, la puerta de enlace local como destino). Es la interfaz Wi-Fi del equipo hacia el router, MCP no interviene en esta capa, es transporte de red local.
Red	Encabezado IPv4 con IP origen 10.115.1.197 (la máquina local, dentro de una red privada) e IP destino 34.143.78.2 (la IP pública asignada por Google al servicio de Cloud Run). El enrutamiento hacia esa IP pública es responsabilidad exclusiva de esta capa.
Transporte	TCP sobre el puerto 443. Se observa el three-way handshake completo (SYN / SYN,ACK / ACK) antes de cualquier byte de TLS, y después ACKs de cada segmento de datos; TCP garantiza entrega ordenada y confiable de los bytes cifrados de TLS y, dentro de estos, de los mensajes JSON-RPC.
Aplicación	Dos protocolos de aplicación coexisten aquí: TLS 1.3 (que en sí mismo negocia claves y cifra todo lo que sigue) y, ya descifrado dentro del túnel TLS, HTTP/1.1 llevando los mensajes JSON-RPC de MCP como cuerpo de cada POST /mcp y de cada respuesta SSE. MCP no define su propio protocolo de transporte de red: reutiliza HTTP como transporte y JSON-RPC como formato de mensaje, todo dentro de esta capa.

Algo interesante es que la respuesta a una solicitud (con id) siempre llega como text/event-stream con un único evento data, incluso cuando el servidor no abre un stream de verdad, el servidor nunca envía mensajes sin que el cliente los pida primero (no hay push del lado del servidor), así que no fue necesario implementar el endpoint GET /mcp que sí contempla la especificación de Streamable HTTP para ese caso.

11. Pruebas y validación

11.1 Suite automatizada

La suite de pruebas (tests/, ejecutada con pytest) contiene 32 casos, todos en estado passing:

- `test_jsonrpc.py`
 - Codificación/decodificación de los cuatro tipos de mensaje JSON-RPC, manejo de errores estándar (-32700, -32600, -32601), generación de ids y framing por línea.
- `test_pharmacy_tools.py`
 - Coincidencia de condiciones por síntomas, señales de alarma (dolor de pecho, menores de 2 años, duración prolongada), bloqueo de interacciones (ibuprofeno + warfarina), conflicto de alergias, rechazo de pedidos con receta faltante o existencias insuficientes, y el recorrido completo del dispatcher (initialize → tools/list → tools/call).
- `test_client_stdio.py`
 - Levanta el servidor Pharmacy como subprocesso genuino y lo recorre por el ciclo de vida MCP completo sobre stdin/stdout reales, sin ningún mock.
- `test_streamable_http_client.py`
 - El transporte StreamableHttpTransport contra un servidor uvicorn real en un socket loopback. Realiza la negociación de sesión, envío/recepción de frames y manejo de errores de conexión.
- `test_http_server.py`
 - El servidor remoto probado por medio de un cliente ASGI, creación y expiración de sesión, notificaciones vs. solicitudes, y los mismos errores JSON-RPC que ya cubría el servidor local.
- `test_transport_resilience.py`
 - Que el cliente y el transporte stdio se recuperan de fallos parciales (conexión cerrada a mitad de handshake, una línea corrupta) sin dejar el proceso en un estado inconsistente.

- `test_agent_loop.py`, `test_anthropic_api.py`, `test_observability.py`
 - Cobertura que antes faltaba del recorrido de tool-calling del agente, del cliente de la API de Anthropic (contra un transporte simulado) y del logger de frames MCP.

11.2 Pruebas manuales

Además de la suite automatizada, se ejecutaron pruebas manuales contra la API real de Anthropic (con clave de API provista por el estudiante) para validar el comportamiento observable, no solo unidades aisladas:

- Conexión simultánea a los tres servidores configurados (filesystem, git, pharmacy) sin errores.
- Triage completo de un caso realista (“rinorrea y dolor de garganta, 24 años, alergia a paracetamol”): el modelo encadenó `assess_symptoms` → `search_medications` → `check_interactions` → `check_stock` y evitó el alérgeno correctamente.
- Caso de señal de alarma (“dolor de pecho y dificultad para respirar”): el servidor forzó `severity urgent` y el modelo se negó a recomendar un medicamento, remitiendo a atención médica inmediata.
- Escenario combinado Filesystem + Git descrito en la sección 3.4, verificado contra el repositorio real con `git log`.
- El REPL real (`python -m src.main`) ejecutado de punta a punta de forma no interactiva, cubriendo `/servers`, `/tools`, `/log` y una conversación completa.

- Verificación bilingüe: el mismo flujo de preguntas y de triage funciona correctamente tanto en español como en inglés, respondiendo siempre en el idioma en que escribe el usuario.
- Conexión y uso end-to-end del servidor remoto desplegado en Cloud Run: el mismo caso de triage (rinorrea, alergia a paracetamol) ejecutado contra pharmacy-remote en vez de la versión local, confirmando que el chatbot usa el servidor remoto exactamente igual que el local, sin ningún cambio de código.
- La interfaz web (python -m src.web.server) probada de punta a punta en el navegador: los tres servidores conectados, cambio de tema en vivo entre el modo general (filesystem/git) y el modo farmacia al invocarse una tool remota, y renderizado correcto de las respuestas del asistente.

12. Dificultades encontradas y cómo se resolvieron

- Falla al lanzar subprocessos con nombres de comando cortos en Windows (FileNotFoundError al invocar npx): CreateProcess de Windows no resuelve shims tipo .cmd por nombre corto de la misma forma en que lo hace una shell. Se resolvió usando shutil.which() para obtener la ruta completa del ejecutable antes de invocar asyncio.create_subprocess_exec en src/mcp/transport/stdio.py.
- UnicodeEncodeError en la consola de Windows: la codificación por defecto de la terminal (cp1252) no puede representar ciertos caracteres que el LLM puede incluir libremente en sus respuestas (marcas de verificación, símbolos, guiones largos), lo que interrumpía el REPL a mitad de una conversación. Se resolvió reconfigurando stdout/stderr a UTF-8 con errors="replace" al iniciar src/main.py.

- La versión publicada del servidor oficial mcp-server-git no incluye una herramienta `git_init` (se confirmó inspeccionando el enum `GitTools` de su código fuente instalado), por lo que “crear un repositorio desde cero” no es posible usando únicamente los dos servidores oficiales. Se documentó como limitación conocida y se ajustó el guion de demostración para inicializar el repositorio de antemano.
- Namespacing de herramientas entre servidores: para permitir que varios servidores expongan tools con el mismo nombre corto sin colisionar, el registro (`src/host/registry.py`) antepone el nombre del servidor (`filesystem_read_file`, `pharmacy_check_stock`) antes de enviarlas al modelo, y revierte el prefijo al enrutar cada llamada de vuelta al cliente MCP correcto.
- Dificultades en el despliegue remoto: el servicio de Cloud Run estaba configurado como "Require authentication" en lugar de "Allow public access" en la consola, lo que hacía que toda petición no autenticada recibiera una página genérica de error de Google en vez de una respuesta de la aplicación. Se corrigió al cambiar la configuración de seguridad en la consola de Cloud Run. Además, se cambió el endpoint `/healthz` por `/status` debido a que el primero está reservado para la infraestructura de Google.

13. Lecciones aprendidas

- Implementar JSON-RPC y el ciclo de vida de MCP a mano (en lugar de usar un SDK) hizo mucho más claro por qué existen ciertas decisiones del protocolo, por ejemplo, por qué `notifications/initialized` no lleva `id` (es una notificación, no espera respuesta),

o por qué `isError` en un resultado de tool es distinto de un error JSON-RPC a nivel de transporte.

- Probar contra servidores MCP oficiales reales, no solo contra el servidor propio, expuso problemas que un entorno completamente controlado no habría revelado, como las particularidades de lanzar subprocesos en Windows o las limitaciones reales del servidor de Git.
- Poner las reglas de seguridad del caso de uso de farmacia en el servidor, y no solo en el prompt del sistema, ayudó mucho, porque el comportamiento en casos de emergencia (negarse a recomendar ante una señal de alarma, exigir receta) no depende de que el modelo decida respetarlo.
- Mantener una separación estricta de capas (protocolo → MCP → LLM → host) facilitó agregar funcionalidad nueva sin tocar código no relacionado. Por ejemplo, el cambio para responder en el idioma del usuario fue una sola línea en el system prompt del host, sin tocar el protocolo ni los servidores.
- Usar Wireshark me ayudó a ver, capa por capa, todo lo que pasa detrás de un mensaje del chatbot. En la capa de enlace vi las direcciones MAC de mi tarjeta de red hacia el router, algo totalmente ajeno a MCP, solo tráfico local. En la capa de red apareció la IP pública real del servidor en Cloud Run, confirmando que el mensaje efectivamente salía a internet.

En la capa de transporte pude ver el handshake TCP (SYN, SYN-ACK, ACK) abriendo la conexión antes de que viajara un solo byte de datos, y después cómo cada segmento se confirmaba con ACKs.

Y en la capa de aplicación entendí que hay dos protocolos trabajando juntos, primero TLS, negociando el cifrado y ya con la conexión cifrada, HTTP llevando dentro los mensajes JSON-RPC de MCP (`initialize`, `tools/call`, etc.).

14. Conclusiones

- Comprendí cómo funciona el protocolo MCP al construirlo desde JSON-RPC puro, sin utilizar un SDK. Para ambas fases del proyecto se utilizó el mismo Dispatcher, los mismos modelos de mensaje y el mismo ciclo de vida. Ambos servidores funcionan igual sobre stdio y sobre Streamable HTTP, cambiando únicamente la capa de transporte, que es exactamente la separación de responsabilidades que el protocolo permite.
- Capturar y descifrar el tráfico con Wireshark me ayudó a comprender que MCP no inventa su propio transporte de red, sino que corre sobre HTTP y TLS igual que cualquier API web. El handshake TCP y TLS que abre la conexión es idéntico al de cualquier sitio HTTPS, lo único propio de MCP es el contenido JSON-RPC que viaja dentro de cada cuerpo de petición y respuesta una vez la conexión está cifrada.
- Agregar la interfaz web como capa adicional sobre el mismo Host confirmó la importancia de implementar una arquitectura modular (por capas). La UI web se conectó al chatbot reutilizando íntegramente el AgentLoop, el ToolRegistry y el hook de logging existentes, sin modificar una sola línea de `src/protocol/` ni de `src/mcp/`.
- El proyecto enseña que gran parte de la dificultad de trabajar con protocolos de red no está en el formato de los mensajes en sí, sino en todo lo que rodea una comunicación real entre dos máquinas distintas: negociación de sesión, control de

acceso, resolución de nombres, cifrado de transporte y las particularidades de la infraestructura de quien aloja el servidor.

15. Referencias

Anthropic. (s. f.). *Messages API reference*. <https://docs.anthropic.com/>

GitHub. (s. f.). *Model Context Protocol servers* [Repositorio de código].
<https://github.com/modelcontextprotocol/servers>

Model Context Protocol. (2025, 25 de noviembre). *Model Context Protocol specification (2025-11-25)*. <https://modelcontextprotocol.io/specification/2025-11-25>

Model Context Protocol. (s. f.). *Architecture*.
<https://modelcontextprotocol.io/docs/learn/architecture>

JSON-RPC Working Group. (s. f.). *JSON-RPC 2.0 specification*.
<https://www.jsonrpc.org/specification>

Google Cloud. (s. f.). Cloud Run documentation. <https://cloud.google.com/run/docs>

Npcap. (s. f.). Npcap: Windows Packet Capture library and driver. <https://npcap.com/>

Wireshark Foundation. (s. f.). TLS decryption. <https://wiki.wireshark.org/TLS>